

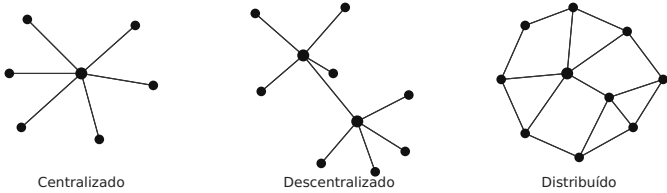
Aula 01 — Introdução (parte 1)

Resumo 2 de 14 · arquivo aula01 · 37 slides · baseado no Cap. 1 de van Steen & Tanenbaum

Esta aula é a introdução “teórica” da disciplina. Ela responde a três perguntas: (1) o que diferencia um sistema **distribuído** de um **descentralizado**; (2) sob quais **aspectos** estudamos sistemas distribuídos; e (3) quais são os **objetivos de projeto** — o que queremos que um bom sistema distribuído alcance. Boa parte da aula detalha esse terceiro ponto.

1. Distribuído × Descentralizado

O senso comum representa três topologias de rede, mas onde termina uma e começa a outra é ambíguo — se um sistema descentralizado vai ganhando conexões, em que momento ele “vira” distribuído?



As três topologias do senso comum. A pergunta provocativa: adicionando $k > 0$ conexões a um descentralizado, quando ele se torna distribuído?

Há **duas visões** para construir um sistema distribuído: a **integrativa** (conectar sistemas em rede já existentes em um sistema maior) e a **expansiva** (estender um sistema existente com computadores adicionais). E há **duas definições** que se distinguem por uma única palavra:

- Sistema **descentralizado**: processos e recursos são *necessariamente* espalhados por múltiplos computadores (muitas vezes cruzando fronteiras administrativas). Ex.: aprendizado de máquina federado, blockchain, monitoramento geograficamente disperso.
- Sistema **distribuído**: processos e recursos são *suficientemente* espalhados por múltiplos computadores. Ex.: servidores de e-mail, redes de distribuição de conteúdo (CDN).

Dois equívocos comuns

- “**Soluções centralizadas não são escaláveis.**” Há diferença entre *logicamente* e *fisicamente* centralizado. A raiz do DNS é logicamente centralizada, fisicamente distribuída e operada por várias entidades.
- “**Ter um ponto único de falha é sempre ruim.**” Nem sempre — pode ser mais fácil de gerenciar e de tornar robusto. A disciplina estuda escalabilidade, tolerância a falhas e segurança justamente para entender esses custos-benefícios.

2. Aspectos dos sistemas distribuídos

São sistemas complexos, estudados sob oito aspectos — que, não por acaso, são os capítulos do curso: **arquitetura** (formas de organizar), **processos** (tipos e relações), **comunicação** (troca de dados), **coordenação** (algoritmos independentes da aplicação), **nomeação** (identificar recursos), **consistência e replicação** (copiar dados para desempenho, mantendo-os consistentes), **tolerância a falhas** (operar apesar de falhas parciais) e **segurança** (controle de acesso e proteção).

3. Objetivos de projeto

Um bom sistema distribuído deve buscar quatro metas gerais: **compartilhamento de recursos**, **distribuição transparente**, **interfaces abertas** (openness) e **escalabilidade**.

3.1 Compartilhamento de recursos

Exemplos clássicos: arquivos na nuvem, streaming peer-to-peer, e-mail compartilhado (ex.: e-mail USP → Gmail), servidores web. Resume-se na frase de John Gage (Sun Microsystems, 1984): “*The network is the computer*” — a rede é o computador.

3.2 Distribuição transparente

Transparência é o esforço do sistema para **mascarar** o fato de que seus processos e recursos estão fisicamente espalhados por várias máquinas, às vezes distantes. Isso costuma ser implementado numa camada entre as aplicações e o sistema operacional: o **middleware**. Existem vários tipos:

Tipo de transparência	O que esconde
Acesso	Diferenças na representação dos dados e no modo de acessar um objeto
Localização	Onde o objeto está fisicamente
Relocação	Que um objeto pode ser movido <i>durante</i> o uso
Migração	Que um objeto pode ser movido de lugar
Replicação	Que existem várias cópias do objeto

Concorrência	Que o objeto é compartilhado por vários usuários ao mesmo tempo
Falha	A ocorrência de uma falha e sua recuperação

Mas **transparência total é cara e nem sempre desejável**: latências de comunicação não podem ser escondidas; esconder falhas de rede/nós é impossível (não dá para distinguir uma resposta lenta de uma falha, nem confirmar se uma operação terminou antes de uma queda); e manter réplicas sempre idênticas custa desempenho. Às vezes **expor a distribuição é benéfico** (serviços por proximidade, fusos horários, estimar quando um servidor caiu). **Conclusão**: transparência é desejável, difícil de atingir e raramente é o objetivo principal.

3.3 Interfaces abertas (openness)

Um **sistema aberto** tem componentes que podem ser facilmente reutilizados ou integrados a outros sistemas. Requer interfaces bem definidas, interoperabilidade, portabilidade de aplicações e facilidade de extensão. Um princípio importante é separar **políticas** (o que se deseja: nível de consistência do cache, o que código de terceiros pode fazer, requisitos de QoS e de segurança) de **mecanismos** (como implementar: configurar cache, níveis de confiança, parâmetros de QoS, algoritmos de criptografia). Quanto mais estrita a separação, mais parâmetros configuráveis surgem — o que dá flexibilidade, mas complica o gerenciamento. Políticas “hard-coded” simplificam, ao custo de flexibilidade.

3.4 Dependabilidade

Um componente **depende** de outro quando a corretude do seu comportamento depende da corretude do outro (componentes são processos ou canais). A dependabilidade tem quatro requisitos:

Requisito	Significado
Disponibilidade	Prontidão para uso (está pronto quando preciso?)
Confiabilidade	Continuidade do serviço ao longo do tempo
Segurança (safety)	Probabilidade muito baixa de catástrofes
Manutenibilidade	Facilidade de consertar após uma falha

Confiabilidade × disponibilidade: a confiabilidade $R(t)$ é a probabilidade de o componente funcionar sem parar em $[0, t)$; disponibilidade é estar pronto num instante. Métricas: **MTTF** (tempo médio até falhar), **MTTR** (tempo médio de reparo) e **MTBF** = MTTF + MTTR (tempo médio entre falhas). A distinção de termos é importante:

Termo	Descrição	Exemplo
Falha (failure)	Componente não cumpre a especificação	Programa trava (crash)
Erro (error)	Parte do componente que leva à falha	Bug de implementação
Defeito (fault)	Causa do erro	Programador desatento

Formas de lidar com defeitos: **prevenção** (evitar que ocorram), **tolerância** (mascarar via redundância), **remoção** (reduzir prevalência/gravidade) e **previsão** (estimar quantidade e consequências).

3.5 Segurança

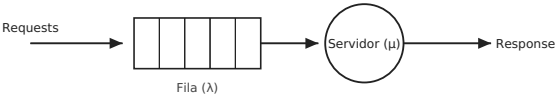
Um sistema sem segurança não oferece dependabilidade. Requisitos: **confidencialidade** (dados só para partes autorizadas) e **integridade** (sem alterações indevidas). Três conceitos-chave: **autenticação** (a identidade é verdadeira?), **autorização** (quais direitos essa identidade tem?) e **confiança** (a entidade se comportará como esperado?). Os mecanismos giram em torno de cifrar/decifrar com chaves — notação $K(\text{dados})$:

- Criptografia simétrica**: a mesma chave secreta cifra e decifra — $\text{dado} = D_K(E_K(\text{dado}))$.
- Criptografia assimétrica**: par de chaves pública (PK) e privada (SK). Para sigilo, Alice cifra com a chave pública de Bob (só Bob, com sua chave privada, lê); para assinar, Alice cifra com a própria chave privada (todos verificam com a pública dela).
- Função de hash $H(\text{dado})$** : gera uma “impressão digital” de tamanho fixo; qualquer mudança no dado muda totalmente o hash, e é inviável achar um dado a partir do hash. Usada para baratear **assinaturas digitais** (assina-se o hash, não a mensagem inteira).

3.6 Escalabilidade

Dizer que um sistema é “escalável” sem qualificar é vago. Existem **três dimensões**: de **tamanho** (nº de usuários/processos), **geográfica** (distância máxima entre nós) e **administrativa** (nº de domínios administrativos). A maioria dos sistemas trata só a de tamanho — servidores independentes em paralelo; escalar geográfica e administrativamente ainda é um desafio.

Os gargalos da escalabilidade de tamanho em soluções centralizadas são **CPU**, **armazenamento** (e a taxa $\text{CPU} \leftrightarrow \text{disco}$) e a **rede** entre usuários e serviço. Isso pode ser modelado como uma **fila simples** (um servidor):



Modelo de fila M/M/1: requisições chegam à taxa λ , o servidor processa à taxa μ .

Definindo a **utilização** $U = \lambda/\mu$ (fração do tempo em que o servidor está ocupado), o resultado mais importante é o **tempo de resposta** (Lei de Little), onde $S = 1/\mu$ é o tempo de serviço:

$$R / S = 1 / (1 - U)$$

A leitura prática: se a utilização U é baixa, o tempo de resposta é quase igual ao tempo de serviço (a requisição quase não espera na fila); mas **à medida que U se aproxima de 100%, o tempo de resposta dispara** — o sistema fica cada vez mais lento. A saída é diminuir S (servir mais rápido).

Escalabilidade geográfica traz outros problemas: migrar de rede local (LAN) para rede ampla (WAN) não é trivial, pois muitos sistemas assumem comunicação síncrona (cliente parado esperando resposta) e a latência atrapalha; conexões WAN perdem mais pacotes; e falta comunicação um-para-muitos (soluções usam serviços de nomeação/diretório, que têm seus próprios limites). A **escalabilidade administrativa** esbarra em políticas conflitantes de uso, pagamento, gerência e segurança (ex.: grids compartilhando recursos entre domínios); uma exceção são redes peer-to-peer como BitTorrent e streaming assistido por pares (Spotify), onde são *usuários* colaborando, não entidades administrativas.

Técnicas para escalabilidade

Quatro técnicas gerais: **(1) ocultar latências** com comunicação assíncrona (nem toda aplicação permite); **(2) aproximar a computação do cliente**; **(3) particionar** dados e computação entre máquinas (ex.: DNS); e **(4) replicar e usar cache**. A técnica (2) é ilustrada com um formulário:

Sem processamento no cliente: cada tecla vai ao servidor (muitas idas e voltas)



Com processamento no cliente: valida local e envia tudo de uma vez (1 ida)



Aproximar a computação do cliente: validar o formulário no navegador (ex.: JavaScript) evita enviar cada tecla ao servidor, reduzindo o tráfego e a carga.

Por fim, a aula alerta sobre os **problemas da replicação** (tema aprofundado depois): múltiplas cópias geram **inconsistência** (ao modificar uma cópia, ela difere das outras); manter todas consistentes exigiria **sincronização global** após cada modificação, o que **impede soluções de larga escala**. A saída pragmática: se a aplicação tolera *alguma* inconsistência, dá para evitar a sincronização global frequente.